# CHAPTER 1

# *Introduction to Expert Systems*

## 1.1 INTRODUCTION

This chapter is a broad introduction to expert systems. The fundamental principles of expert systems are introduced. The advantages and disadvantages of expert systems are discussed and the appropriate areas of application for expert systems are described. The relationship of expert systems to other methods of programming are discussed.

## 1.2 WHAT IS AN EXPERT SYSTEM?

The first step in solving any problem is defining the problem area or **domain** to be solved. This consideration is just as true in artificial intelligence (AI) as in conventional programming. However, because of the mystique formerly associated with AI, there is a lingering tendency to still believe the old adage "It's an AI problem if it hasn't been solved yet." Another popular definition is that "AI is making computers act like they do in the movies." This type of mind set may have been popular in the 1970s when AI was entirely in a research stage. However, today there are many real-world problems that are being solved by AI and many commercial applications of AI.

Although general solutions to classic AI problems such as natural language translation, speech understanding, and vision have not been found, restricting the problem domain may still produce a useful solution. For example, it is not difficult to build simple natural language systems if the input is restricted to sentences of the form noun, verb, and object. Currently, systems of this type work well in providing a user-friendly interface to many software products such as database systems and spreadsheets. In fact, the parsers associated with popular computer text-adventure games today exhibit an amazing degree of ability in understanding natural language.

As Figure 1.1 shows, AI has many areas of interest. The area of **expert systems** is a very successful approximate solution to the classic AI problem of programming intelligence. Professor Edward Feigenbaum of Stanford University, an early pioneer of expert systems technology, has defined an expert system as "an intelligent computer program that uses knowledge and inference procedures to solve problems that are difficult enough to require significant human expertise for

their solutions." (Feigenbaum 82). That is, an expert system is a computer system that **emulates** the decision-making ability of a human expert. The term *emulate* means that the expert system is intended to act in all respects like a human expert. An emulation is much stronger than a simulation, which is only required to act like the real thing in some respects.

Although a general-purpose problem solver still eludes us, expert systems function well in their restricted domains. As proof of their success, you need only observe the many applications of expert systems today in business, medicine, science, and engineering, as well as all the books, journals, conferences, and products devoted to expert systems.

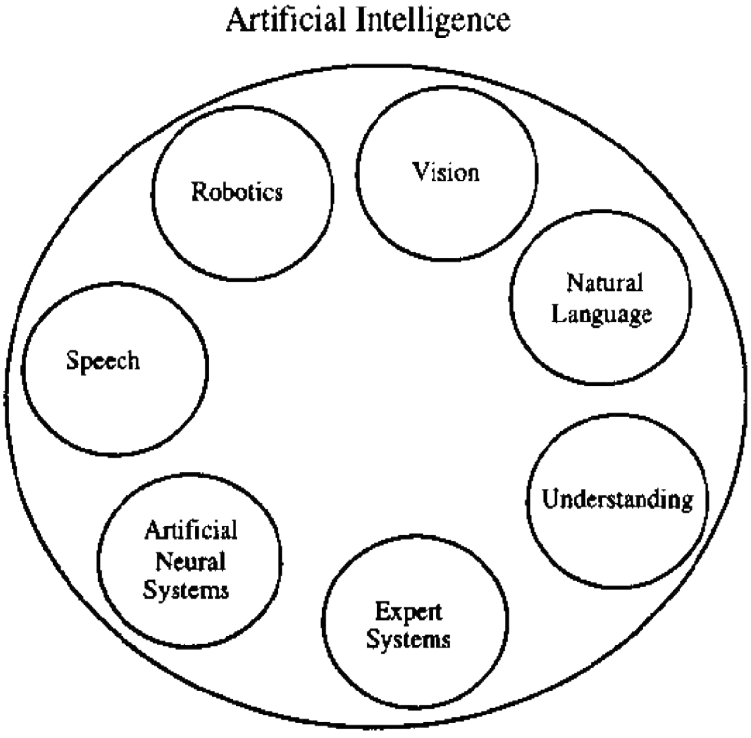



**Figure 1.1 Some Areas of Artificial Intelligence**

Expert systems is a branch of AI that makes extensive use of specialized knowledge to solve problems at the level of a human **expert**. An expert is a person who has **expertise** in a certain area. That is, the expert has knowledge or special skills that are not known or available to most people. An expert can solve problems that most people cannot solve or can solve them much more efficiently (but not as cheaply). When expert systems were first developed in the 1970s, they contained expert knowledge exclusively. However, the term *expert system* is often applied today to any system that uses expert system technology. This expert system technology may include special expert system languages, programs, and hardware designed to aid in the development and execution of expert systems.

The knowledge in expert systems may be either expertise or knowledge that is generally available from books, magazines, and knowledgeable persons. The terms *expert system*, **knowledge-based system**, or **knowledge-based expert system** are often used synonymously. Most people use the term expert system simply because it's shorter, even though there may be no expertise in their expert system, only general knowledge.

Figure 1.2 illustrates the basic concept of a knowledge-based expert system. The user supplies facts or other information to the expert system and receives expert advice or **expertise** in response. Internally, the expert system consists of two main components. The knowledge base contains the knowledge with which the **inference engine** draws conclusions. These conclusions are the expert system's responses to the user's queries for expertise.

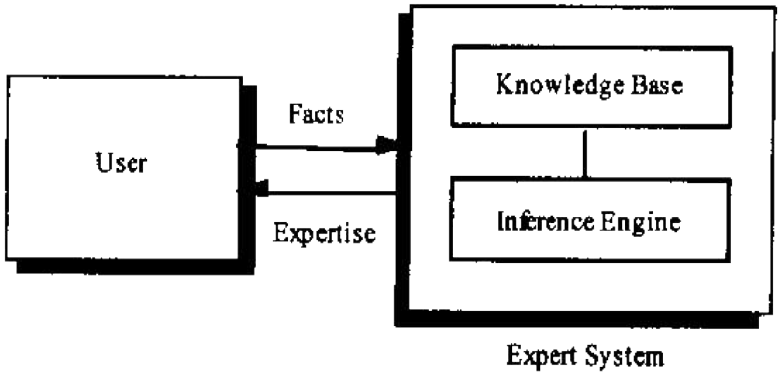



**Figure 1.2 Basic Concept of an Expert System Function**

Useful knowledge-based systems have also been designed to act as an intelligent assistant to a human expert. These intelligent assistants are designed with expert systems technology because of the development advantages. As more knowledge is added to the intelligent assistant, it acts more like an expert. Thus, developing an intelligent assistant may be a useful milestone in producing a complete expert system. In addition, it may free up more of the expert's time by speeding up the solution of problems. Intelligent tutors are another new application of artificial intelligence. Unlike the old computer-assisted instruction systems, the new systems can provide context-sensitive instruction (Giarratano 91a).

An expert's knowledge is specific to one **problem domain**, as opposed to knowledge about general problem-solving techniques. A problem domain is the special problem area such as medicine, finance, science, or engineering, and so forth that an expert can solve problems in very well. Expert systems, like human experts, are generally designed to be exports in one problem domain. For example, you would not normally expect a chess expert to have expert knowledge about medicine. Expertise in one problem domain does not automatically carry over to another.

The expert's knowledge about solving specific problems is called the **knowledge domain** of the expert. For example, a medical expert system designed to diagnose infectious diseases will have a great deal of knowledge about certain symptoms caused by infectious diseases. In this case, the knowledge domain is medicine and consists of knowledge about diseases, symptoms, and treatments. Figure 1.3 illustrates the relationship between the problem and knowledge domain. Notice that this knowledge domain is entirely included within the problem domain. The portion outside the knowledge domain symbolizes an area in which there is not knowledge about all the problems.

One medical expert system usually does not have knowledge about other branches of medicine such as surgery or pediatrics. Although its knowledge of iofectious disease is equivalent to a human expert, the expert system would not

know anything about other knowledge domains unless it was programmed with that domain knowledge.

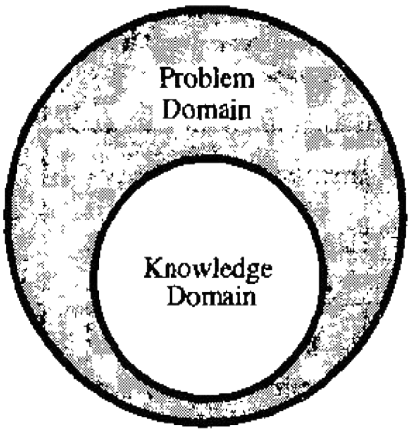



**Figure 1.3 A Possible Problem and Knowledge Domain Relationship**

In the knowledge domain that it knows about, the expert system reasons or makes **inferences** in the same way that a human expert would infer the solution of a problem. That is, given some facts, a conclusion that follows is inferred. For example, if your spouse hasn't spoken to you in a month, you may infer that he or she had nothing worthwhile to say. However, this is only one of several possible inferences.

As with any new technology, we still have a lot to learn about expert systems. Table 1.1 summarizes the differing views of the participants in a technology. In this table, the technologist may be an engineer or software designer and the technology may be hardware or software. In solving any problem, these are questions that need to be answered or the technology will not be successfully used. Like any other tool, expert systems have appropriate and inappropriate applications. As our experience with expert systems grows, we will discover what these applications are.

| Person | Question |
|---|---|
| Manager | What can I use it for? |
| Technologist | How can I best implement it? |
| Researcher | How can I extend it? |
| Consumer | How will it help me? |
| | Is it worth the trouble and expense? |
| | How reliable is it? |

**Table 1.1 Differing Views of Technology**

## 1.3 ADVANTAGES OF EXPERT SYSTEMS

*Expert systems have a number of attractive features:*

- *Increased availability.* Expertise is available on any suitable computer hardware. In a very real sense, an expert system is the mass production of expertise.

- *Reduced cost.* The cost of providing expertise per user is greatly lowered.
- *Reduced danger.* Expert systems can be used in environments that might be hazardous for a human.
- *Permanence.* The expertise is permanent. Unlike human experts, who may retire, quit, or die, the expert system's knowledge will last indefinitely.
- *Multiple expertise.* The knowledge of multiple experts can be made available to work simultaneously and continuously on a problem at any time of day or night. The level of expertise combined from several experts may exceed that of a single human expert (Harmon 85).
- *Increased reliability.* Expert systems increase confidence that the correct decision was made by providing a second opinion to a human expert or break a tie in case of disagreements by multiple human experts. Of course, this method probably won't work if the expert system was programmed by one of the experts. The expert system should always agree with the expert, unless a mistake was made by the expert. However, this may happen if the human expert was tired or under stress.
- *Explanation.* The expert system can explicitly explain in detail the reasoning that led to a conclusion. A human may be too tired, unwilling, or unable to do this all the time. This increases the confidence that the correct decision is made.
- *Fast response.* Fast or real-time response may be necessary for some applications. Depending on the software and hardware used, an expert system may respond faster and be more available than a human expert. Some emergency situations may require responses faster than a human and so a real-time expert system is a good choice (Hugh 88; Ennis 86).
- *Steady, unemotional, and complete response at all times.* This may be very important in real-time and emergency situations, when a human expert may not operate at peak efficiency because of stress or fatigue.
- *Intelligent tutor.* The expert system may act as an intelligent tutor by letting the student run sample programs and by explaining the system's reasoning.
- *Intelligent database.* Expert systems can be used to access a database in an intelligent manner (Kerschberg 86; Schur 88).

The process of developing an expert system has an indirect benefit also since the knowledge of human experts must be put into an explicit form for entering into the computer. Because the knowledge is then explicitly known instead of being implicit in the expert's mind, it can be examined for correctness, consistency, and completeness. The knowledge may then have to be adjusted or re-examined, which improves the quality of the knowledge.

## 1.4 GENERAL CONCEPTS OF EXPERT SYSTEMS

The knowledge of an expert system may be represented in a number of ways—it can be encapsulated in rules and objects. One common method of representing knowledge is in the form of IF . . . THEN type–**rules**, such as

```
IF the light is red THEN stop
```

If a fact exists that the light is red, this matches the pattern "the light is red." The rule is satisfied and performs its action of "stop." Although this is a very simple example, many significant expert systems have been built by expressing the knowledge of experts in rules. In fact, the knowledge-based approach to developing expert systems has completely supplanted the early AI approach of the 1950s and 1960s, which tried to use sophisticated reasoning techniques with no reliance on knowledge. Some types of expert system tools such as CLIPS allow **objects** as well as rules. Rules can pattern match on objects as well as facts. Alternatively, objects can operate independently of the rules.

Today, a wide range of knowledge-based expert systems have been built. Large systems containing thousands of rules, such as the XCON/R1 system of Digital Equipment Corporation, know much more than any single human expert on how to configure computer systems (McDermott 84). Many small systems for specialized tasks have also been constructed with several hundred rules. These small systems may not operate at the level of an expert, but are designed to take advantage of expert systems technology to perform knowledge-intensive tasks. For these small systems the knowledge may be in books, journals, or other publicly available documentation.

In contrast, a classic expert system embodies unwritten knowledge that must be extracted from an expert by extensive interviews with a **knowledge engineer** over a long period of time. The process of building an expert system is called ***knowledge engineering*** and is done by a knowledge engineer (Michie 73). Knowledge engineering refers to the acquisition of knowledge from a human expert or other source and its coding in the expert system.

The general stages in the development of an expert system are illustrated in Figure 1.4. The knowledge engineer first establishes a dialog with the human expert in order to elicit the expert's knowledge. This stage is analogous to a system designer in conventional programming discussing the system requirements with a client for whom the program will be constructed. The knowledge engineer then codes the knowledge explicitly in the knowledge base. The expert then evaluates the expert system and gives a critique to the knowledge engineer. This process iterates until the system's performance is judged by the expert to be satisfactory.

The expression ***knowledge-based system*** is a better term for the application of knowledge-based technology, which may be used for the creation of either expert systems or knowledge-based systems. However, like the term artificial intelligence, it is common practice today to use the term expert systems when referring to both expert systems and knowledge-based systems, even when the knowledge is not at the level of a human expert.

Expert systems are generally designed differently from conventional programs because the problems usually have no algorithmic solution and rely on inferences to achieve a reasonable solution. Note that a reasonable solution is about the best we can expect if no algorithm is available to help us achieve the optimum solution. Because the expert system relies on inference, it must be able to explain its reasoning so that its reasoning can be checked. An **explanation facility** is an integral part of sophisticated expert systems. In fact, elaborate explanation facilities may be designed to allow the user to explore multiple lines of "what if" or **hypothetical reasoning** questions and even to translate natural language into rules.

Some expert systems even allow the system to learn rules by example, through **rule induction**, in which the system creates rules from tables of data. Formalizing the knowledge of experts into rules is not simple, especially when the expert's knowledge has never been systematically explored. There may be inconsistencies, ambiguities, duplications, or other problems with the expert's knowledge that are not apparent until attempts are made to formally represent the knowledge in an expert system.

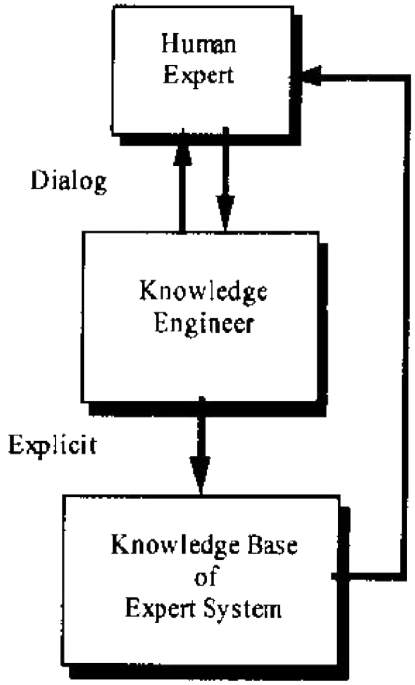



**Figure 1.4 Development of an Expert System**

Human experts also know the extent of their knowledge and qualify their advice as the problem reaches their **limits of ignorance**. A human expert also knows when to "break the rules." Unless expert systems are explicitly designed to deal with uncertainty, they will make recommendations with the same confidence even if the data they are dealing with are inaccurate or incomplete. An expert system's advice, like that of a human expert, should degrade gracefully at the boundaries of ignorance.

A practical limitation of many expert systems today is lack of **causal knowledge**. That is, the expert systems do not really have an understanding of the underlying causes and effects in a system. It is much easier to program expert systems with **shallow knowledge** based on empirical and heuristic knowledge than with **deep knowledge** based on the basic structures, functions, and behaviors of objects. For example, it is much easier to program an expert system to prescribe an aspirin for a person's headache than to program all the underlying biochemical, physiological, anatomical, and neurological knowledge about the human body. The programming of a causal model of the human body would be an enormous task and, even if successful, the response time of the system would probably be extremely slow because of all the information the system would have to process.

One type of shallow knowledge is **heuristic knowledge** (*heuristic* is Greek and means "to discover"). Heuristics are not guaranteed to succeed in the same way that an algorithm is a guaranteed solution to a problem. Instead, heuristics are

rules of thumb or empirical knowledge gained from experience that may aid in the solution but are not guaranteed to work. However, in many fields, such as medicine and engineering, heuristics play an essential role in some types of problem solving. Even if an exact solution is known, it may be impractical to use because of cost or time constraints. Heuristics can provide valuable shortcuts that can reduce both time and cost.

Another problem with expert systems today is that their expertise is limited to the knowledge domain that the systems know about. Typical expert systems cannot generalize their knowledge by using **analogy** to reason about new situations the way people can. Although rule induction helps, only limited types of knowledge can be put into an expert system this way. The customary way of building an expert system, by having the knowledge engineer repeat the cycle of interviewing the expert, constructing a prototype, testing, interviewing, and so on, is a time-consuming and labor-intensive task. In fact, this problem of transferring human knowledge into an expert system is so major that it is called the ***knowledge acquisition bottleneck***. This is a descriptive term because the knowledge acquisition bottleneck constricts the building of an expert system like an ordinary bottleneck constricts fluid flow into a bottle.

In spite of their present limitations, expert systems have been successful in dealing with real-world problems that conventional programming methodologies have been unable to solve, especially those dealing with uncertain or incomplete information. The important point is to be aware of the advantages and limitations of this new technology so that it can be appropriately utilized.

## 1.5 CHARACTERISTICS OF AN EXPERT SYSTEM

An expert system is usually designed to have the following general characteristics:

- *High performance*. The system must be capable of responding at a level of competency equal to or better than that of an expert in the field. That is, the quality of the advice given by the system must be very high.
- *Adequate response time*. The system must also perform in a reasonable amount of time, comparable to or better than the time required by an expert to reach a decision. An expert system that takes a year to reach a decision compared to an expert's time of one hour would not be too useful. The **time constraints** placed on the performance of an expert system may be especially severe in the case of real-time systems, when a response must be made within a certain time interval.
- *Good reliability*. The expert system must be reliable and not prone to crashes or it will not be used.
- *Understandable*. The system should be able to explain the steps of its reasoning while executing so that it is understandable. Rather than being just a "black box" that produces a miraculous answer, the system should have an explanation capability in the same way that human experts can explain their reasoning. This feature is very important for several reasons.

One reason is that human life and property may depend on the answers of the expert system. Because of the great potential for harm, an expert system must be

able to justify its conclusions in the same way a human expert can explain why a certain conclusion was reached. Thus, an explanation facility provides an understandable check of the reasoning for humans.

A second reason for having an explanation facility occurs in the development phase of an expert system to confirm that the knowledge has been correctly acquired and is being correctly used by the system. This is important in debugging because the knowledge may be incorrectly entered by typos or be incorrect due to misunderstandings between the knowledge engineer and the expert. A good explanation facility allows the expert and the knowledge engineer to verify the correctness of the knowledge. Also, because of the way typical expert systems are constructed, it is very difficult to read a significant program listing and understand its operation.

An additional source of error may be unforeseen interactions in the expert system, which may be detected by running test cases with known reasoning that the system should follow. As we will discuss in more detail later, multiple rules may apply to a given situation about which the system is reasoning. The flow of execution is not sequential in an expert system so that you cannot just read its code line by line and understand how the system operates. That is, the order in which rules have been entered in the system is not necessarily the order in which they will be executed. The expert system acts much like a parallel program in which the rules are independent knowledge processors.

- *Flexibility.* Because of the large amount of knowledge that an expert system may have, it is important to have an efficient mechanism for adding, changing, and deleting knowledge. One reason for the popularity of rule-based systems is the efficient and modular storage capability of rules.

Depending on the system, an explanation facility may be simple or elaborate. A simple explanation facility in a rule-based system may list all the facts that made the latest rule execute. More elaborate systems may do the following:

- *List all the reasons for and against a particular hypothesis.* A hypothesis is a goal that is to be proved, such as "the patient has a tetanus infection" in a medical diagnostic expert system. In a real problem there may be multiple hypotheses, just as a patient may have several diseases at once. A hypothesis can also be viewed as a fact whose truth is in doubt and must be proved.
- *List all the hypotheses that may explain the observed evidence.*
- *Explain all the consequences of a hypothesis.* For example, assuming that the patient does have tetanus, there should also be evidence of fever as the infection runs its course. If this symptom is then observed, it adds credibility that the hypothesis is true. If the symptom is not observed, it reduces the credibility of the hypothesis.
- *Give a prognosis or prediction of what will occur if the hypothesis is true.*
- *Justify the questions that the program asks of the user for further information.* These questions may be used to direct the line of reasoning to likely diagnostic paths. In most real problems it is too expensive or takes too long to explore all possibilities, and some way

must be provided to guide the search for the correct solution. For example, consider the cost, time, and effect of administering all possible medical tests to a patient complaining of a sore throat.

- *Justify the knowledge of the program.* For example, if the program claims that the hypothesis "the patient has a tetanus infection" is true, the user could ask for an explanation. The program might justify this conclusion on the basis of a rule that says that if the patient has a blood test that is positive for tetanus, then the patient has tetanus. Now the user could ask the program to justify this rule. The program could respond by stating that a blood test positive for a disease is proof of the disease.

In this case, the program is actually quoting a **metarule**, which is knowledge about rules (the prefix *meta* means "above" or "beyond"). Some programs such as Meta-DENDRAL, have been explicitly created to infer new rules (Buchanan 78). A hypothesis is justified by knowledge, and the knowledge is justified by a **warrant** that it is correct. A warrant is essentially a meta-explanation that explains the expert system's explanation of its reasoning.

Knowledge can easily grow **incrementally** in a rule-based system. That is, the knowledge base can grow little by little as rules are added so that the performance and correctness of the system can be continually checked. If the rules are properly designed, the interactions between rules will be minimized or eliminated to protect against unforeseen effects. The incremental growth of knowledge facilitates **rapid prototyping** so that the knowledge engineer can quickly show the expert a working prototype of the expert system. This is an important feature because it maintains both the expert's and management's interest in the project. Rapid prototyping also quickly exposes any gaps, inconsistencies, or errors in the expert's knowledge or the system so that corrections can be made immediately.

## 1.6 THE DEVELOPMENT OF EXPERT SYSTEMS TECHNOLOGY

AI has many branches concerned with speech, vision, robotics, natural language understanding and learning, and expert systems. The roots of expert systems lie in many disciplines. In particular, one of the major roots of expert systems is the area of human information processing called **cognitive science**. *Cognition* is the study of how humans process information. In other words, cognition is the study of how people think, especially when solving problems.

The study of cognition is very important if we want to make computers emulate human experts. Often experts can't explain how they solve problems—the solutions just come to them. If the expert can't explain how a problem is solved, it's not possible to encode the knowladge in an expert system based on explicit knowledge. In this case the only possibility is programs that learn by themselves to emulate the expert. These are programs based on induction and artificial neural systems, which will be discussed later.

### Human Problem Solving and Productions

The development of expert systems technology draws on a wide background. Table 1.2 is a brief summary of some important developments that have converged